Lecture 6: Admin Features (Academic Registrar) CSC 1202: Software Development Project

Peter Wakholi (Ph.D)

SCIT Makerere University

Learning Objectives

- By the end of this lecture, you should be able to:
  - View and filter logged issues in the admin portal.
  - Assign issues to lecturers or mark them as resolved.
  - Implement notifications for status changes.
  - Develop and consume API endpoints (in Django) for issue management.

Recap of Previous Week

- What we covered previously:
  - Student Features:
    - Issue submission form and validations
    - Student dashboard to track issue status
  - Django API endpoint for storing issue data
- Why the Registrar Role Matters:
  - Central authority to coordinate resolution
  - Needs advanced features (filtering, assigning, etc.)

The Registrar Dashboard – Overview

- Key Functions:
  - **View All Issues:** List of all issues logged by students.
  - **Filter Issues:** By category (missing marks, appeals, corrections), status (open, assigned, resolved), or student ID.
  - **Assign/Resolve:** Assign an issue to a specific lecturer or department, or mark it as resolved.
  - **Notifications:** Trigger notifications to relevant parties on assignment or resolution.

Viewing and Filtering Logged Issues

- Approach:
  - React Table (or similar) to display issues with columns (Issue ID, Student, Type, Status, etc.).
  - Search/Filter Inputs: Filter by text, dropdowns, or advanced queries.
- Database & Backend: Use Django QuerySets for filtering.
- Pass filtered data to the frontend via a REST endpoint.

Demo: Simple Filtering in Django

```python
# views.py
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Issue
from .serializers import IssueSerializer

@api_view(['GET'])
def filter_issues(request):
    status = request.GET.get('status', None)
    issues_qs = Issue.objects.all()
    if status:
        issues_qs = issues_qs.filter(status=status)
    serializer = IssueSerializer(issues_qs, many=True)
    return Response(serializer.data)
```

Assigning Issues to Lecturers

- Workflow:
  - Registrar selects an issue.
  - Chooses a lecturer from a list.
  - System updates the issue’s assigned_to field.
  - Notification is sent to the lecturer and student.
- Front-End (React):
  - A dropdown or modal for selecting a lecturer.
  - Update button triggers an API call (PUT or PATCH) to the Django backend.

Marking Issues as Resolved

- Steps:
  - Admin checks if an issue is complete.
  - Clicks “Mark Resolved” button in React interface.
  - Status changed to “Resolved” in the database.
  - Student gets a notification or sees “resolved” in their dashboard.
- Implementation:
  - PUT or PATCH request to the issue endpoint:

Implementing Notifications

- Why Notifications?
  - Keeps all parties informed of changes (students, lecturers, registrar).
  - Enhances user experience and ensures timely actions.
- Possible Approaches:
  - Email Notifications (Django’s built-in email features).
  - In-App Notifications (Real-time updates with websockets or periodic checks).
  - SMS (Third-party service like Twilio, optional for advanced projects).

Sample Notification Flow

- Flow Diagram:
  1. Registrar updates an issue status to "Assigned".
  2. Django triggers a signal or function to send an email to the assigned lecturer and the student.
  3. React front-end receives an update or can poll the API for changes.
  4. Registrar sees a confirmation “Notification Sent” or a success toast.

Assignment Instructions

- Develop a module for the Registrar to:
  - View a list of issues.
  - Filter issues by type, status, or assigned lecturer.
- Implement Issue Forwarding:
  - A feature to assign or reassign an issue to a lecturer.
- Add “Resolve Issue” button and logic:
  - Mark an issue as resolved.
  - Trigger a simple notification (email or console log if email is not yet configured).

Programming Task Details

- Frontend (React):
  - Use a table or list to display issues (e.g., React Table).
  - Provide filtering options in the UI.
  - Provide controls (buttons or modals) for assigning and resolving issues.
- Backend (Django):
  - Create or update endpoints to handle:
    - GET /issues (with optional query params for filtering).
    - PATCH /issues/<id>/assign or issues/<id>/resolve.
  - Implement any necessary notifications.

Tips & Best Practices

- **Separate Concerns:** Keep the assignment/resolve logic on the backend, not the frontend.
- **Security:** Verify the requesting user is the Registrar before allowing modifications.
- **Validation:** Ensure an issue can only be “resolved” if it has been assigned or is open.
- **Performance:** Use QuerySet filters effectively in Django to avoid heavy database load.